The first key concept we introduce is the class of *equivalence mappings*, Φ, for a given Dec-POMDP.

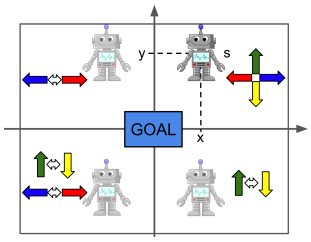

Figure 2.

Each element of Φ is a bijection of each of $\mathcal{S}$, O, and $\mathcal{A}$ onto itself, such that it leaves the Dec-POMDP unchanged:

$$\begin{aligned}\phi \in \Phi \iff & P(\phi(s')|\phi(s),\phi(a)) = P(s'|s,a) \\ & \wedge R(\phi(s'),\phi(a),\phi(s)) = R(s',a,s) \\ & \wedge O(\phi(o)|\phi(s),\phi(a),i) = O(o|s,a,i) \\ & \text{where equalities apply } \forall s', s, a'\end{aligned}$$

In other words, Φ describes the symmetries in the underlying Dec-POMDP. We note that our notation is heavily overloaded since each ϕ can act on actions, states and the observation function, so ϕ shorthand for $\phi = \{\phi_{\mathcal{S}}, \phi_{\mathcal{A}}, \phi_O\}$.

Next, we extend ϕ to also act on trajectories:

$$\phi(\tau_t^i) = \{\phi(o_0^i), \phi(a_0^i), \phi(r_0), \cdots, \phi(o_t^i)\}.$$

At this point an example might be helpful: consider a grid-world with a robot, shown in Figure 2, that can move in the 4 cardinal directions. In our example the goal is in the middle of the room, which leaves two axis of symmetry: We can invert either the x-axis, the y-axis or both, as long as we make the corresponding changes to the action space, for example mapping "up" to "down" and vice versa when inverting the y-axis.

In a similar way, we can extend ϕ to act on policies π, as follows:

$$\pi' = \phi(\pi) \iff \pi'(\phi(a)|\phi(\tau)) = \pi(a|\tau), \forall \tau, a$$

These symmetries are the "payoff irrelevant" parts of the Dec-POMDP. They come from the fact that the actions and states in the Dec-POMDP do not come with labels and so taking a policy and permuting it with respect to these symmetries does not change the outcome of interest: the trajectory and the reward.

It is precisely these symmetries that can cause problems for self-play trained agents. Since agents are trained together, they can coordinate on how to break symmetries. However, there is no guarantee that multiple SP agents trained separately will break symmetries in the same way. In this case when they are paired together their policies may fail spectacularly.

The goal of OP, then, will be to build policies which are maximally robust to this failure mode.

4. Other Play

We consider the 2 agent case for ease of notation with π^1, π^2 denoting each agent's component of the policy and π denoting the joint policy.

First, consider self-play (SP) learning rule. This is the learning rule which tries to optimize the following objective:

$$\pi^* = \arg\max_{\pi} J(\pi^1, \pi^2) \tag{1}$$

When the Dec-POMDP is tabular we can solve this via various methods. When it is not, deep reinforcement learning (deep RL) can be used to apply function approximation and gradient based optimization of this objective function. Though there is a large literature focusing on various issues in multi-agent optimization (Busoniu et al., 2006; Hernandez-Leal et al., 2019), our paper is agnostic to the precise method used.

These policies can be arbitrary and in complicated Dec-POMDPs multiple maxima to Equation 1 will often exist. These multiple policies can (and, as we will see in our experiments, often will) use coordinated symmetry breaking to receive high payoffs. Therefore, 2 matched, separately trained, SP agents will not necessarily receive the same payoff with each other as they receive with themselves.

To alleviate this issue, we need to make the optimization problem more robust to the symmetry breaking.

Let us consider the point of view of constructing a strategy for agent 1 where agent 2 will be the unknown novel partner. The *other-play* (OP) objective function for agent 1 maximizes expected return when randomly matched with a symmetry-equivalent policy of agent 2 rather than with a particular one. In other words, we perform a version of self-play where agents are not assumed to be able to coordinate on exactly how to break symmetries.

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\phi\sim\Phi}\ J(\pi^1, \phi(\pi^2)) \tag{2}$$

Here the expectation is taken with respect to a uniform distribution on Φ. We call this expected return J_{OP}. We will now consider what policies maximize J_{OP}.

Lemma 1.

$$J(\pi_A, \pi_B) = J(\phi(\pi_A^1), \phi(\pi_B^2)), \quad \forall \phi \in \Phi, \pi_A, \pi_B$$

This Lemma follows directly from the fact that the MDP is invariant to any $\phi \in \Phi$.

Lemma 2.

$$\{\phi \cdot \phi' : \phi' \in \Phi\} = \Phi\ ,\ \forall \phi \in \Phi$$

This Lemma follows from the fact that Φ is a bijection.

Proposition 1. *The expected OP return of π is equal to the expected return of each player independently playing a policy π^i_Φ which is the uniform mixture of $\phi(\pi^i)$ for all $\phi \in \Phi$.*

Proof.

$$J_{OP}(\pi) = \mathbb{E}_{\phi\sim\Phi}\ J(\pi^1, \phi(\pi^2)) \tag{3}$$
$$= \mathbb{E}_{\phi_1\sim\Phi,\phi_2\sim\Phi}\ J(\phi_1(\pi^1), \phi_1(\phi_2(\pi^2))) \tag{4}$$
$$= \mathbb{E}_{\phi_1\sim\Phi,\phi_2\sim\Phi}\ J(\phi_1(\pi^1), \phi_2(\pi^2)) \tag{5}$$
$$= J(\pi_\Phi) \tag{6}$$

(4) follows from Lemma 1, (5) follows from Lemma 2.

□

Corollary 1. *The distribution π^*_{OP} produced by OP will be the uniform mixture π_Φ with the highest return $J(\pi_\Phi)$.*

Let $\mathcal{L}_i$ be the set of learning rules which input a Dec-POMDP and output a policy for agent i.

A meta-equilibrium is a learning rule for each agent such that neither agent can improve their expected payoff by unilaterally deviating to a different learning rule.

Proposition 2. *If agent i uses OP as their learning rule then OP is a payoff maximizing learning rule for the agent's partner. Furthermore both agents using OP is the best possible meta-equilibrium.*

Proof. Since the Dec-POMDP has no labels for actions and states, $\mathcal{L}_i$ must choose all $\phi(\pi)$ with equal probability. Among these possible outputs, π^*_{OP} maximizes the return by Corollary 1. □

5. Implementing Other Play via Deep RL

We now turn to optimizing the OP objective function. In many applications of interest the Dec-POMDP is not tabular. Thus, deep RL algorithms use function approximation for the state space and attempt to find local maxima of Equation 1 using self-play reinforcement learning.

We show how to adapt this method to optimize the other-play objective (Equation 2). This amounts to applying a very specific kind of asymmetric domain randomization (Tobin et al., 2017) during training.

During each episode of MARL training, for each agent i a random permutation $\phi_i \in \Phi$ is chosen uniformly iid from Φ, and agent i observes and acts on $\phi(\mathcal{S}, O, \mathcal{A})$. Importantly, the agents act in different permutations of the same environment.

This environment randomization is equivalent to other-play, because the MDP remains constant under ϕ_i while the effect of agent i's policy on the environment is $\phi_i(\pi_i)$. The fixed points of independent optimization of π under this learning rule will be joint policies where each π_i is a best response to the uniform mixture of permutations of partner policies, i.e. precisely the permutation-invariant equilibria that are the solutions of other-play.

We note that OP is fundamentally compatible with any type of optimization strategy and can be applied whenever there are symmetries in the underlying MDP.

6. Experiments

We evaluate OP in two different settings. In each setting we will compare standard SP against OP. We will perform comparisons of agents trained together to agents trained separately that are placed into a zero-shot coordination test game.

6.1. Lever Game

We begin with the "lever game" mentioned in the introduction. This environment is tabular, there are only 10 actions possible per player. Here, during training, we use simple joint action learning, compute the true gradient with respect to the current policy and update. We show training time (i.e. expected reward with itself) and test time (zero-shot) coordination performance for both SP (optimizing equation 1) and OP (optimizing equation 2). The code is available as a notebook online here and can be executed online without downloading: `https://bit.ly/2vYkfI7`.

Figure 3 shows the results. As expected, OP agents coordinate on the unique option of 0.9 points both during the training phase and at test time. As a consequence, OP agents can carry out successful zero-shot coordination when paired with other OP agents.

In contrast, SP agents achieve higher rewards of 1.0 points during the training phase but entirely fail to coordinate with other, independently trained, SP agents.

6.2. Hanabi with AI Agents

We now turn to a much more complex environment. We construct agents for the cooperative card game Hanabi, which has recently been established a benchmark environment for multi-agent decision making in partially observable settings (Bard et al., 2020).

Hanabi is a cooperative card game with the interesting twist that players cannot see their own cards and hence have to rely on receiving information from the other player (who can see their hand). In Hanabi, there are two main ways of exchanging information: first of all, players can take

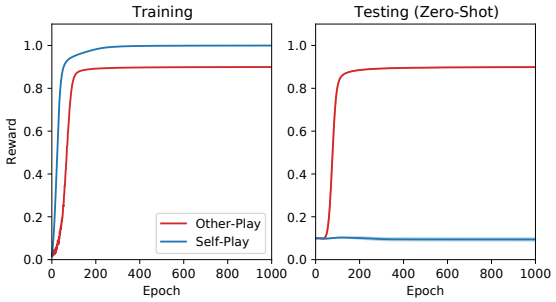

Figure 3. Train and test performance of self-play and other-play algorithms on the lever coordination game. Shown is the mean, shading is the standard error of the mean (s.e.m.), across 30 different seeds.

costly "hint" actions that point out subsets of cards based on rank or color. For example, hinting for "blue" reveals the color of all blue cards. Secondly, observing the actions themselves can be informative, in particular when players have pre-established conventions. The goal in Hanabi is to play cards in a legal order completing stacks of cards, one for each color. There are 5 color and 5 cards and the maximum points is 25. Players will lost a life token if they play a card out of order. Once they exhaust the deck or lose all 3 lives ("bomb out"), the game will terminate.

As noted, the vast majority of research on Hanabi has been in the self-play setting, in which a group of agents is trained to jointly obtain the highest possible score. To apply OP in Hanabi we note that assuming no side information, a permutation of the colors of the cards leaves the game unchanged. We use this as our class of symmetries.

6.3. MARL Training Details

OP can be applied on top of any SP algorithm. In Hanabi the Simplified Action Decoder (SAD) (Hu & Foerster, 2019) method achieves SOTA performance for RL agents. We use SAD as a base algorithm onto which we add OP. We use the open-sourced implementation of SAD as well as most of its hyper-parameters but with two major modifications. First, we use 2 GPUs for simulation instead of 1 as in the original paper. This doubles the data generation speed and has a profound effect on reducing the wall clock time required to achieve competitive performance. Second, we introduce extra hyper-parameters that control the network architecture to add diversity to the model capacity in order to better demonstrate the effectiveness of OP. Specifically, the network can have either 1 or 2 fully connected layers before 2 LSTM layers and can have an optional residual connection to by-pass the LSTM layers. For SP, we re-train the base SAD and the SAD + AUX variant proposed in Hu & Foerster (2019). SAD + AUX is specifically engineered for Hanabi by adding an auxiliary task to predict whether a card is playable, discardable, or unknown. We train agents with the aforementioned 4 different network architectures. We run each hyper-parameter configuration with 3 different seeds and thus 12 models are produced for each category of {SAD, SAD + AUX, SAD + OP, SAD + AUX + OP}.

6.4. Evaluation

We evaluate the models within the same category by pairing different models together to play the game, a process we refer to as *cross-play*. Clearly, if independent training runs ("seeds") from the same training method fail to coordinate with each other at test time it is unlikely they will coordinate with agents optimized by through a different process, let alone humans. As such, cross-play is a cheap proxy to evaluate whether a training method has potential for zero-shot coordination with human players. Figure 4 shows the scores obtained between all pairs of agents. Table 1 shows the average within-pair and cross-play scores. We see that SAD coordinates with itself but fails to coordinate with any other SAD agent. SAD with OP, however, significantly improves the cross-play. The effect is especially profound when the model has limited representation power. The top left corner of the graph, which corresponds to the simplest models that have only 1 fully connected layers, 2 LSTM layers and no residual connection, shows almost perfect cooperation scores. With the network growing more complicated, different strategies start to emerge and the cross-play performance drops. Auxiliary task implicitly improves cross-play scores by encouraging all agents to act basing on grounded information and confident predictions. Nonetheless, adding OP to SAD + AUX further improves performance and achieves the highest cross-play payoffs.

We can further study the policies resulting from these learning algorithms. Figure 5 picks the agent with highest cross-play performance in each category (top row) as well as their worst possible partner (bottom row) and presents $P(a_t^i \mid a_{t-1}^j)$ over a subset of actions averaged over timesteps in 1000 episodes generated through self-play. In other words, we ask, do the agents respond very differently to possible actions of their partner? A large difference indicates that what an agent would do in a situation is very different from what their partner would do: a recipe for miscoordination!

We see that two paired SAD agents have very different policies and thus miscoordinate a lot. They also learn "inhuman" conventions that are hard for human to understand. For example, the agent hints Color5 to indicate discarding the 1st card while its partner interprets that as playing the 2nd card. OP eliminates these type of conventions. From the plot and our experience of playing with the SAD + OP agent, we find that it tends to use color hints to indicate either that the partner should save the card, or to disambiguate with a